

第一章 概述

1. 通常把软件生命周期全过程中使用的一整套技术方法的集合称为软件工程方法学，它包含方法、工具和过程三个要素。常用的软件工程方法包括结构化方法和面向对象方法。(p1)

2. 结构化技术开发的软件，具有稳定性差、可维护性差和可复用性差的缺点。(p2)

3. 面向对象软件工程的主要优点 (p4-5)

① 符合常规的思维方式。面向对象软件工程把问题域的概念映射到对象及其关系，符合人们通常的思维方式，避免了结构化软件工程从问题域到分析阶段的映射问题。

② 连续性好。面向对象软件工程的分析、设计和编码采用一致的模型表示，各阶段文档表示一致，后一阶段可直接利用前一阶段的结果，避免了结构化软件工程各阶段表示方法不连续的问题（数据流图与模块结构图需要转换），降低了理解的难度，减少了工作量并降低了映射误差。另外，程序与问题域一致，发现了程序中的错误，可以方便地逆向追溯到问题域；需求发生了变化，可以方便地从问题域正向追踪到程序。

③ 可维护性好。面向对象方法具有封装性，可以使对象结构在外部功能发生变化后保持相对稳定，并使改动局限于一个对象的内部。修改一个对象时，很少影响其他对象，避免了波动效应。具体来说，两方面的原因决定了维护对象是容易的。首先，概念独立性（封装）意味着很容易判断出产品的哪一部分必须进行修改，以达到某一确定的维护目标——无论是对完善性维护还是对纠错性维护。其次，信息隐藏确保了对象本身的修改不会在该对象以外产生影响，因此大大降低了回归错误的数量。

④ 可复用性好。面向对象方法的继承性和封装性可以很好地支持软件复用，并易于扩充。

4. 应该考简答题：软件设计与体系结构的发展阶段

软件体系结构发展的第一个阶段是“无体系结构”设计阶段，以汇编语言进行小规模应用程序开发为特征。第二个阶段是萌芽阶段，出现了程序结构设计主题，主要特征是使用了控制流图和数据流图。第三个阶段是初期阶段，出现了从不同侧面描述系统的结构模型，以统一建模语言（Unified Modeling Language, UML）为典型代表。第四个阶段是高级阶段，以描述系统的高层抽象结构为中心，不关心具体的建模细节，划分了体系结构模型与传统的软件结构的界限。(p5)

第二章 面向对象方法

1. 用例图也称用例模型，描述的是外部执行者(Actor)所理解的系统功能。用例模型用于需求分析阶段 (p7)

2. 用例图包含系统、执行者和用例三种模型元素。(p8)

3. 在用例图中除了包含执行者与用例之间的关联外，还有常用的两种类型的关联，用以表示用例之间的包含和扩展关联。(p9)

4. 类图是用于显示出类、接口以及它们之间的静态结构和关系的元素。(p10)

5. 类图中的关系：一般化关系、关联关系、聚合关系、合成关系、依赖关系 (p11-13)

6. 广义上讲，组件是具有某种功能的可复用的软件结构单元，是为组装服务的，是组成软件系统的计算单元或数据存储单元。(p15, 简答)

7. 连接(Connection)是组件间建立和维护行为关联与信息传递的途径。(p16)

8. 常见的软件开发模型大致可分为三种类型：① 瀑布模型，软件需求完全确定；② 渐进式开发模型(如螺旋模型)，软件开发初始阶段只能提供基本需求；③ 变换模型，以形式化开发方法为基础。

9. 并发指宏观(从应用程序开发者层次上看，时间尺度较大)上计算机可以同时执行多个不相关的工作任务(是并行的)，但从微观(从操作系统的线程管理角度和计算机硬件工程师层次)上时间尺度很小角度来看，这些工作任务并不是始终都在运行，每个工作任务都呈现出走走停停这种相互交替的状态。(p44)

第三章 经典软件体系结构风格

1. 主程序-子过程风格的**优点**是有效地将一个较复杂的程序系统设计任务，分解成许多易于控制和处理的子任务，便于开发和维护；已被证明是成功的设计方法，可以被用于较大程序。缺点是在程序规模方面，程序超过 10 万行表现不好，程序太大时开发太慢、测试越来越困难；可重用性差、数据安全性差，难以开发大型软件和图形界面的应用软件；把数据和处理数据的过程分离为相互独立的实体，当数据结构改变时，所有相关的处理过程都要进行相应的修改；图形用户界面的应用程序，很难用过程来描述和实现，开发和维护也很困难。(p53, 简答)

2. 面向对象风格具有抽象、封装、继承、多态等特点；(p55)

3. 3 种典型的数据流风格是批处理 (Batch Sequential)、管道 - 过滤器 (Pipe-Filter) 和过程控制 (Process Control)。本节主要阐述批处理风格和管道 - 过滤器风格。(p57)

4. 管道/过滤器风格与批处理风格的相似点是都把任务分解成为一系列固定顺序的计算单元，而且彼此间只通过数据传递交互。它们的不同点在于，批处理风格是整体传递和处理数据、组件粒度大、延迟高、实时性差、无并发，管道/过滤器风格是增量处理、组件粒度较小、实时性好、可并发。(p60, 简答)

5. 组件持续地与其所处的环境打交道，但并不知道确切的交互次序，这时需要采用**隐式调用**。(p62)

6. 基于层次风格软件体系结构的系统的优点：(p68, 老师说不用全背)

① 支持基于抽象程度递增的系统设计，一个复杂问题的求解可以被分解为一系列递增的步骤。

② 良好的可扩展性。因为每一层一般只能和相邻的上下层交互，功能的改变最多影响相邻的上下层，所以系统易于改进和扩大。

③ 每一层的软件都支持复用。每一层可以定义一组对外的标准接口，而同一层内允许各种不同的实现方法。只要提供的服务接口定义不变，同一层的不同实现就可以交换使用。

④ 可替换性，只要提供的服务接口定义不变，同一层的不同实现可以交换使用。这样，就可以定义一组标准的接口，而允许各种不同的实现方法。

⑤ 对标准化的支持。清晰定义并且广泛接受的抽象层次能够促进实现标准化的任务和接口开发，同样接口的不同实现能够互换使用。

⑥ 可测试性。具有定义明确的层接口，以及交换层接口的各个实现的能力，提高了可测试性。

但是，基于层次风格软件体系结构的系统也有其不足之处：

① 并不是每个系统都可以很容易地划分为分层的模式，即使一个系统的逻辑结构是层次化的，由于对系统性能的考虑，系统设计师也不得不把一些低级或高级的功能综合起来。

② 效率的降低。由分层风格构成的系统，运行效率往往低于整体结构。在上层中的服务如果有很多依赖于最底层，则相关的数据必须通过一些中间层的若干次转化，才能传到。

③ 很难找到合适的、正确的层次抽象方法。层数太少，分层不能完全发挥这种风格的可复用性、可更改性和可移植性上的潜力。层数过多，则引入不必要的复杂性和层间隔离冗余以及层间传输开销。目前，没有可行的广为人们所认可的层粒度的确定和层任务的分配方法。

层次风格常用于通信协议。最著名的层次风格的软件体系结构的例子，是国际标准化组织（ISO）的 OSI（Open System Interconnection，开放系统互连参考模型）分层通信模型；其他的典型例子还包括操作系统、数据库系统等。

7. 第七层是应用层（Application Layer），为数据提供各种各样的收发方式，为网络用户提供各种应用服务（文件传输、电子邮件、分布式数据库、网络管理等）。

8. 黑板体系结构是仓库体系结构的特殊化。它反映的是一种信息共享的系统，如同教室里的黑板一样，有多个人读也有多个人写。

黑板系统通常由三部分组成：

① 知识源：软件专家模块，每个知识源提供应用程序所需要的具体的专家知识。知识源之间不直接进行通信，只通过黑板来完成它们之间的交互。

② 黑板数据结构：一个共享知识库，包含了问题、部分解决方案、建议和已经贡献的信息。它按照与应用程序相关的层次来组织解决问题的数据。黑板可以被认为是一个动态的“库”，知识源通过不断地改变黑板数据来解决问题。

③ 控制机制：知识源需要控制机制来保证以一种最有效和连贯的方式来工作，控制完全由黑板的状态驱动，黑板状态的改变决定使用的特定知识。（p71-72，简答）

9. 在具体构造一个专家系统时，除了应该具有这几部分之外，还应根据相应领域问题的特点及要求适当增加某些部件。

如：人机接口、知识获取机构（学习系统）、知识库及其管理系统、推理机、数据库及其管理系统、解释机构（p73-74）

10. 解释器风格中有四个组件：一个状态机和三个存储器。一个状态机是完成解释工作的解释引擎；第一个存储器是伪码的数据存储区——正在被解释的程序，第二个存储器记录源代码被解释执行的进度（被解释的程序的状态），第三个存储器记录解释引擎当前工作状态。（p74，简答）

11. 开环控制系统和闭环控制系统的区别：（p76）

① 从工作原理上看，开环控制系统的输出量不对系统的控制产生任何影响；闭环控制系统的输出量返回到输入端并对控制系统过程产生影响。

② 从信息传递过程上看，开环控制系统信息路径一条，自输入端传至输出

端，不存在信息逆向流动；闭环控制系统信息路径两条，一条自输入端传至输出端，另一条是输出端信息经反馈环节返回到输入端的比较环节，形成一个闭合的环路。

基于反馈控制环风格软件体系结构的系统要达到以下要求：

- ① 闭环控制要消除过程中的干扰作用 —— 定值控制、干扰控制。
- ② 一个过程的被控量必须始终准确地跟随变动的给定值 —— 跟踪控制、随动控制。

另外，还有一种自适应过程控制环，它包括三方面的工作：辨识被控对象的特征；在辨识的基础上做出控制决策；在决策的基础上实施修正动作。

第四章 分布式软件体系结构风格

1. 两层 C/S 体系结构的**优点**如下：

① 两层 C/S 体系结构具有强大的数据操作和事务处理能力，模型思想简单，易于人们理解和接受。

② 系统的客户应用程序和服务器组件分别运行在不同的计算机上，系统中每台服务器都可以适合各组件的要求，这对于硬件和软件的变化显示出极大的适应性和灵活性，而且易于对系统进行扩充和缩小。

③ 在两层 C/S 体系结构中，系统中的功能组件充分隔离，客户应用程序的开发集中于数据的显示和分析，而数据库服务器的开发则集中于数据的管理。将大的应用处理任务分布到许多通过网络连接的低成本计算机上，以节约大量费用。

两层 C/S 体系结构虽然具有一些优点，但随着企业规模的日益扩大，软件的复杂程度不断提高，**两层 C/S 体系结构逐渐暴露出以下缺点：**

① 互操作性差。采用不同开发工具或平台开发的软件，一般互不兼容，不能或很难移植到其他平台上运行。使用 DBMS 所提供的私有的数据编程语言来开发业务逻辑，降低了 DBMS 选择的灵活性，导致软件移植困难，新技术无法轻易使用。例如，Oracle DB 提供的若干存储过程函数，在 SQL Server 上就不能执行。

② 系统管理与配置成本高。采用两层 C/S 体系结构的软件要升级，开发人员必须到现场为客户机升级，每个客户机上的软件都需要维护。当系统升级时，对软件的一个小小的改动，每一个客户端都必须更新，导致软件维护和升级困难。

③ 系统伸缩性差。用户数增加到一定数量时，性能急剧恶化，服务器成为系统的瓶颈。

④ 开发成本较高。两层 C/S 体系结构对客户端软硬件配置要求较高，尤其是软件的不断升级，对硬件要求不断提高，增加了整个系统的成本，且客户端变得越来越臃肿。

⑤ 客户端程序设计复杂。采用两层 C/S 体系结构进行软件开发，客户端的程序设计要进行大量工作，客户端显得十分庞大。

⑥ 用户界面风格不一，使用繁杂，不利于推广使用。

(p82. 考选择题)

2. P2P 体系结构风格把客户机和服务器看成是相对的。根据实际需要，系统中的每一台计算机既可以作为客户机，又可以作为服务器，即每一台计算机既可以请求其他结点提供的服务，又可以为其他结点提供服务。当它请求其他结点

的服务时，它就是客户机；当它向其他结点提供服务时，它就是服务器。（p92，简答题）

3. 目前，P2P 类业务主要有如下 4 类：P2P 下载类业务，如 Bittorrent、eMule、迅雷 P2P 下载等；P2P 流媒体类业务，如 PPLive、QQLive、PPSteam 等；IM（Instant Messaging，即时通信）类业务，如 QQ 等；VoIP（Voice over Internet Protocol，网络电话 / IP 电话）类业务，如 Skype、H.323、SIP、MGCP 等。（p92，选择题）

4. 三层 C/S 体系结构可以将应用功能分成表示层、功能层、和数据层三部分（P94）

5. B/S 体系结构风格有以下优点：（p96-97 考察选择题）

① 基于 B/S 体系结构的软件，系统安装、修改和维护全在服务器端解决，系统维护成本低。客户端无任何业务逻辑，用户在使用系统时，仅需要一个浏览器就可运行全部的模块，真正达到了“零客户端”的功能，很容易在运行时自动升级；良好的灵活性和可扩展性，对于环境和应用条件经常变动的情况，只要对业务逻辑层实施相应的改变，就能够达到目的。

② B/S 体系结构还提供了异种机、异种网、异种应用服务的联机、联网、统一服务的最现实的开放性基础。

③ 较好的安全性。在这种结构中，客户应用程序不能直接访问数据，应用服务器不仅可控制哪些数据被改变和被访问，而且还可控制数据的改变和访问方式。

④ 真正意义上的“瘦客户端”，从而具备了很高的稳定性、延展性和执行效率。

⑤ 可以将服务集中在一起管理，统一服务于客户端，从而具备了良好的容错能力和负载平衡能力。

⑥ 扩大了组织计算机应用系统功能覆盖范围，可以更加充分利用网络上的各种资源，同时应用程序维护的工作量也大幅减少。B/S 结构出现之前，管理信息系统的功能覆盖范围主要是组织内部；B/S 结构“零客户端”方式使组织的供应商和客户（这些供应商和客户有可能是潜在的，也就是说可能是事先未知的）的计算机方便地成为管理信息系统的客户端，进而在限定的功能范围内查询组织相关信息，完成与组织的各种业务往来的数据交换和处理工作。

⑦ B/S 结构的计算机应用系统与 Internet 的结合也使新近提出的一些新的企业计算机应用（如电子商务、客户关系管理）的实现成为可能。

B/S 体系结构风格也存在一些缺点：

① 客户端浏览器以同步的请求 / 响应模式交换数据，每请求一次服务器就要刷新一次页面。

② 受 HTTP 协议“基于文本的数据交换”的限制，在数据查询等响应速度上，要远远低于 C/S 体系结构。

③ 数据提交一般以页面为单位，数据的动态交互性不强，不利于在线事务处理（OLTP）应用。

④ 受限于 HTML 的表达能力，难以支持复杂 GUI（如报表等）。

因此，虽然 B/S 结构的计算机应用系统有很多优越性，但由于 C/S 结构的计算机应用系统网络负载较小，未来一段时间内将是 B/S 结构和 C/S 结构共存的情况。但是，计算机应用系统计算模式的发展趋势是向 B/S 结构转变。

6. （C/S 和 B/S 混合软件体系结构）这种混合体系软件结构的缺点是：外部

用户可以直接访问数据库服务器，企业数据容易暴露给外部用户，无法保障数据库的安全（p100）

第五章 MVC 风格与 Struts 框架

1. MVC 风格将交互式应用划分为模型、视图和控制器 3 种组件，连接件是显示调用、隐式调用、其他协议（如 HTTP 协议）（p107-108，简答吧）
2. 控制器接收用户的输入并调用模型和视图去完成用户的需求（p108，选择）

第六章 软件设计目标

1. （p125，简答：简述软件设计的主要目标）

本章主要讲述软件设计的正确性、健壮性、可复用性、可维护性和高效性目标。

① 正确性：指设计符合需求，代码按照要求执行。通常对给定的需求可能有多种正确的设计。

② 健壮性：如果在出现错误时应用程序也能执行，那么设计是具有健壮性的；错误的种类很多，用户在操作应用程序时会出现一些错误，软件开发者在设计和编码时也会犯一些错误。

③ 可复用性：指可以将其方便地移植到其他应用中。

④ 可维护性：一个可维护性的设计意味着可以很容易地对其进行修改。

⑤ 高效性：有两个方面 —— 时间效率和空间效率。

2. （p133，简答）

软件系统的性能要求都是会变化的，系统的设计应该为日后的变化留出足够的空间。一个好的系统设计应该有如下性质：可扩展性、灵活性、可插入性。

① 可扩展性：新的性能可以很容易地加入系统中。要加入一个新性能，在增加一个独立的新模块的同时，不会影响很多其他模块，不会造成几个模块的改动。例如，一个新的制动防滑系统应该可以在不影响汽车其他部分的情况下加入系统中。如果加入这个防滑系统之后，汽车的转向盘因此出现问题，那么这个系统就不是扩展性好的系统。

② 灵活性：代码修改不会波及很多其他的模块。例如，一辆汽车的空调发生了故障，技师把空调修好之后，系统的发动机不能启动了，这就不是一个很灵活的系统。

③ 可插入性：可以很容易地将一个类用另一个有同样接口的类代替。一段代码、函数、模块可以在新的模块或者新系统中使用，这些已有的代码不会依赖于一大堆其他的东西。例如，应该可以很容易地将一辆汽车的防撞气囊取出来，换上一个新的。如果将气囊取出来后，汽车的传动杆不工作了，那么这个系统就不是一个可插入性很好的系统。

第七章 设计原则

1. 里氏代换原则（Liskov Substitution Principle, LSP）是指所有引用基类（父类）的地方必须能透明地使用其子类的对象。子类型必须能够替换掉它们的父类型；子类继承了父类，子类可以以父类的身份出现。（p146）

2. 单一职责原则 (Single Responsibility Principle, SRP) 即功能要单一, 一个对象应该只包含单一的职责, 并且该职责被完整地封装在一个类中。或者说, 就一个类而言, 应该仅有一个引起它变化的原因。(p165)

3. 接口隔离原则 (Interface Segregation Principle, ISP) 的第一种解释是客户端不应该依赖那些它不需要的接口。另一种解释是一旦一个接口太大, 则需要将它分割成一些更细小的接口, 使用该接口的客户端仅需要知道与之相关的方法即可。(p166)